

CO3 AT1 SET 3

Computer Architecture 1220

1. Processor Datapath for Register Transfers and Logical Operations

Introduction

A processor data path is the part of a CPU that performs data movement and processing. It contains registers, ALU, buses, multiplexers and control signals. Register transfer operations move data between registers, while the ALU performs arithmetic and logical operations.

Basic Datapath



Register Transfer

A register transfer means moving data from one register to another.

Example: $R2 \leftarrow R1$

Execution sequence:

1. Select R1 as the source register.
2. Enable the output of R1.
3. Transfer the data through the bus.
4. Enable the load signal of R2.
5. Data is stored in R2.

XOR Operation

For:

$R3 \leftarrow R1 \text{ XOR } R2$

Execution:

1. Read R1 and R2.
2. Send both values to the ALU.
3. Set ALU control signal to XOR.
4. ALU performs XOR.
5. Store result in R3.

Example:

R1 = 1010

R2 = 1100

XOR 0110

Therefore,

R3 = 0110

NOT Operation

For:

$R2 \leftarrow \text{NOT } R1$

The ALU receives R1 and performs the NOT operation.

Example:

R1 = 1010

NOT R1 = 0101

Comparison Operation

For:

Compare R1 and R2

The ALU subtracts or compares the two values and sets appropriate **status flags**, such as:

- Zero Flag (Z)
- Carry Flag (C)
- Sign/Negative Flag (S)
- Overflow Flag (V)

If $R1 = R2$, the **Zero Flag becomes 1**.

Control Signals

Operation	Main Control Signal
Register selection	Register Select
Register loading	Load/Write Enable
Bus transfer	Bus Enable
XOR	ALU = XOR
NOT	ALU = NOT
Comparison	ALU = COMPARE
Result storage	Register Write

Conclusion

The processor data path performs register transfers and logical operations under the control of control signals. Proper coordination between registers, buses and ALU allows the processor to execute instructions efficiently.

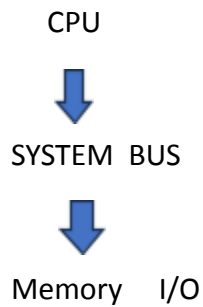
2. Bus-Based System for Processor, Memory and I/O

Introduction

A bus is a communication pathway used to transfer data between the CPU, memory and I/O devices. A computer system generally uses three main buses:

1. **Data Bus**
2. **Address Bus**
3. **Control Bus**

Basic Bus Organization



1. Data Bus

The data bus transfers actual data.

Example:

CPU → Memory

or

I/O → CPU

The width of the data bus determines how much data can be transferred at a time.

2. Address Bus

The address bus carries the address of the memory location or I/O device.

For example:

CPU → Address Bus → Memory

A larger address bus can access a larger memory space.

3. Control Bus

The control bus carries control signals such as:

- Read
- Write
- Memory Enable
- I/O Enable

- Interrupt
- Clock signals

Bus Width and Performance

A wider bus can transfer more bits per cycle.

For example:

- 8-bit bus → 8 bits per transfer
- 16-bit bus → 16 bits per transfer
- 32-bit bus → 32 bits per transfer
- 64-bit bus → 64 bits per transfer

Therefore, increasing bus width generally improves data transfer capacity.

Transfer Rate

Bus transfer rate depends on:

Transfer Rate = Data transferred per cycle × Number of cycles per second

A higher bus frequency increases the transfer rate.

Bus Contention

Bus contention occurs when two or more devices try to use the same bus at the same time.

It can cause:

- Delays
- Reduced performance
- Data conflicts

Bus arbitration techniques are used to decide which device gets control of the bus.

Conclusion

A bus-based system provides an efficient communication path between the CPU, memory and I/O devices. Bus width, transfer rate and contention directly affect the overall system performance.

3. Pipelined Processor and Effect of Pipeline Depth

Introduction

Pipelining is a processor technique in which instruction execution is divided into multiple stages. Different instructions can be processed simultaneously in different stages.

A common five-stage pipeline is:

IF → ID → EX → MEM → WB

Where:

- **IF** – Instruction Fetch
- **ID** – Instruction Decode
- **EX** – Execute
- **MEM** – Memory Access
- **WB** – Write Back

Pipeline Example

Suppose there are 4 instructions:

1 2 3 4 5 6

I1 IF ID EX MEM WB

I2 IF ID EX MEM WB

I3 IF ID EX MEM WB

I4 IF ID EX MEM WB

Instead of waiting for one instruction to completely finish, the processor starts another instruction while the previous one moves to the next stage.

Arithmetic Instruction

Example:

ADD R1, R2, R3

The ALU performs:

R1 = R2 + R3

Memory Instruction

Example:

LOAD R1, [1000]

The processor fetches data from memory location 1000 and stores it in R1.

Branch Instruction

Example:

BEQ R1, R2, LABEL

If R1 and R2 are equal, the processor branches to LABEL.

Effect of Pipeline Depth

Increasing pipeline depth divides instruction execution into more stages.

Advantages:

- Higher clock frequency
- Better instruction throughput
- More instructions completed per unit time

Disadvantages:

- More pipeline registers
- Increased complexity

Speedup

Theoretical pipeline speedup can approach the number of pipeline stages.

For a 5-stage pipeline, ideal speedup can approach:

Speedup ≈ 5

Conclusion

Pipelining improves processor performance by increasing instruction throughput. Increasing pipeline depth can improve clock speed, but excessive depth increases complexity and hazard penalties.

4. Data and Control Hazards in a Pipeline

Introduction

A pipeline hazard is a situation that prevents the next instruction from executing normally. The three major types are:

1. Structural hazard
2. Data hazard
3. Control hazard

This question mainly focuses on data and control hazards.

Data Hazard

A data hazard occurs when one instruction depends on the result of an earlier instruction.

Example:

I1: ADD R1, R2, R3

I2: SUB R4, R1, R5

Here, I2 needs the value of **R1** produced by I1.

Problem

I1: ADD → produces R1

I2: SUB → needs R1

I2 may try to read R1 before I1 has written the result.

Solution 1: Stalling

The processor temporarily stops the dependent instruction.

I1: IF ID EX MEM WB

I2: IF ID ST ST EX MEM WB

Advantage: Simple and safe.

Disadvantage: Reduces performance.

Solution 2: Forwarding

The result is directly sent from one pipeline stage to another without waiting for it to be written back to the register.

ALU Result —————▶ Next ALU Input

Forwarding reduces unnecessary stalls.

Control Hazard

A control hazard occurs mainly because of branch instructions.

Example:

BEQ R1, R2, LABEL

The processor does not immediately know whether the branch will be taken.

Solution 1: Stalling

The processor waits until the branch decision is known.

Solution 2: Branch Prediction

The processor predicts whether the branch will be taken.

Two common types:

- **Static branch prediction**
- **Dynamic branch prediction**

Performance Impact

Technique	Performance
Stalling	Lower
Forwarding	Better
Branch prediction	Higher

Good prediction + forwarding Very high

Conclusion

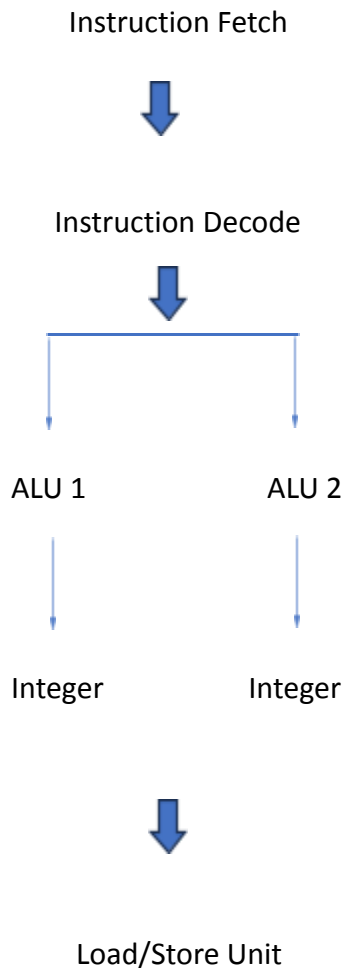
Data and control hazards reduce pipeline performance. Stalling, forwarding and branch prediction are important techniques used to minimize these delays and improve processor throughput.

5. Superscalar Processor with Multiple Execution Units and Out-of-Order Processing

Introduction

A superscalar processor can execute more than one instruction in a single clock cycle. It contains multiple execution units such as ALUs, floating-point units and load/store units.

Basic Structure



Multiple Execution Units

A superscalar processor may contain:

- Integer ALU
- Floating Point Unit

- Load/Store Unit
- Branch Unit

Therefore, independent instructions can execute simultaneously.

Example:

I1: ADD R1, R2, R3

I2: SUB R4, R5, R6

Since both instructions are independent:

Cycle 1 → I1 and I2 can be issued

Cycle 2 → Both can execute

This increases instruction throughput.

Out-of-Order Execution

In out-of-order execution, instructions do not necessarily execute in the same order in which they were fetched.

Example:

I1: LOAD R1, Memory

I2: ADD R2, R3, R4

I3: SUB R5, R1, R6

I3 depends on I1, so it must wait.

But I2 is independent. Therefore:

I1 → Waiting for memory

I2 → Execute immediately

I3 → Wait for I1

This avoids keeping the execution units idle.

Instruction Scheduling

Instruction scheduling selects instructions that are ready for execution.

It considers:

- Data dependencies
- Available execution units
- Instruction priority
- Resource availability

Resource Allocation

The processor allocates different execution units according to instruction requirements.

Example:

Instruction Required Unit

ADD	Integer ALU
MUL	Multiply/Floating Unit
LOAD	Load Unit
STORE	Store Unit
Branch	Branch Unit

Efficient resource allocation prevents execution units from remaining idle.

Benefits

1. Multiple instructions can execute simultaneously.
2. Instruction throughput increases.
3. Processor resources are utilized efficiently.
4. Delays caused by memory operations can be reduced.
5. Independent instructions can execute while another instruction is waiting.
6. Overall processor performance improves.

Conclusion

Superscalar processors improve performance by using multiple execution units and executing independent instructions in parallel. Out-of-order execution, instruction scheduling and efficient resource allocation help maximize instruction throughput and processor utilization.

★ Quick Summary for Revision

Question	Main Concept	Important Points
1	Processor Datapath	Registers, ALU, XOR, NOT, comparison, control signals
2	Bus Organization	Data bus, address bus, control bus, width, transfer rate, contention
3	Pipelining	IF, ID, EX, MEM, WB, throughput, speedup, pipeline depth
4	Pipeline Hazards	Data hazard, control hazard, stalling, forwarding, branch prediction
5	Superscalar Processor	Multiple units, parallel execution, out-of-order execution, scheduling

These are written to fit the **5-mark Level-4/Exemplary style** shown in your rubric, while keeping the language easy to study and write in the assignment.